

SOFTWARE SECURITY

Summer Semester 2026

June 2026

Secure Messaging System Using Encryption Techniques

Project and Report

Submitted to:

Prof. Dr. Rand Kouatly

Submitted by:

Ashok Kumar Meena

University of Applied Sciences Europe

Software Security Homework

ABSTRACT

This project presents a Python-based Secure Messaging System that demonstrates essential software security principles in a practical way. The application allows users to register, log in, and exchange encrypted messages through a command-line interface. User passwords are protected using PBKDF2-HMAC-SHA256 with a unique random salt for every account. Messages are encrypted using AES-GCM, and message integrity is verified using HMAC-SHA256 before decryption. The system also includes a logging mechanism and attack simulations such as man-in-the-middle message tampering and brute-force login attempts. The implementation proves how confidentiality, integrity, and authentication can be applied in a simple secure messaging application.

1. INTRODUCTION

Modern communication systems must protect private information from unauthorized access, modification, and misuse. When messages are transmitted or stored without encryption, attackers can read, change, or replay sensitive information. To address these risks, this project implements a secure messaging system using symmetric encryption, password hashing, and message authentication techniques.

The developed system is a local Python CLI application. It focuses on the core security concepts required by the assignment instead of unnecessary user-interface complexity. The application provides authenticated access, encrypted message storage, message integrity verification, attack detection, and security event logging.

2. OBJECTIVES

Design and implement a secure messaging application using Python.

Provide user registration and login with securely hashed passwords.

Encrypt all messages using AES symmetric encryption before storage.

Verify message integrity using HMAC-SHA256.

Prevent decryption when a message has been modified.

Simulate security attacks such as MITM tampering and brute-force login attempts.

Record important security events in logs.

Provide a simple command-line interface and clear documentation.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Any modern laptop or desktop computer.

Minimum 4 GB RAM recommended.

At least 100 MB free disk space for source code, dependencies, and local database.

3.2 Software Requirements

Python 3.10 or above.

cryptography library for AES-GCM encryption.

sqlite3 for local database storage.

hashlib and hmac for password hashing and integrity verification.

getpass for secure password/shared-secret input.

secrets for generating cryptographically secure random salts and nonces.

pytest for running the automated test suite.

Visual Studio Code or any Python IDE.

Terminal or command prompt for running the CLI application.

4. SYSTEM DESIGN

The Secure Messaging System is divided into small modules so that each security function is easy to understand and test.

Module	Purpose
User Authentication Module	Handles user registration, login, password hashing, and session handling.
Database Module	Stores users, encrypted messages, and security logs in SQLite.
Cryptography Module	Derives keys, encrypts messages using AES-GCM, and verifies HMAC-SHA256.
Messaging Module	Allows logged-in users to send encrypted messages and read decrypted inbox messages.
Attack Simulation Module	Demonstrates MITM/tampering and brute-force login attempts.
Logging Module	Records security-relevant events such as login attempts, encryption, decryption failure, and attack detection.

4.1 Application Flow

User starts app

- > Register or Login
- > Session is created after successful login
- > User sends message to another registered user
- > Shared secret is entered and converted into encryption/HMAC keys
- > Message is encrypted using AES-GCM
- > HMAC-SHA256 is generated over message data
- > Encrypted message is stored in SQLite
- > Receiver enters shared secret
- > HMAC is verified before decryption
- > If verification passes, plaintext is shown
- > If verification fails, tampering is detected and logged

5. IMPLEMENTATION

5.1 Project File Structure

```
secure_messaging_system/  
app.py - CLI menu and session handling  
auth.py - registration, login, password hashing  
crypto_utils.py - AES-GCM encryption + HMAC integrity  
database.py - SQLite storage (users, messages, logs)  
logger_utils.py - security logging  
attack_simulation.py - MITM / tampering simulation  
demo_data.py - demo users (alice, bob) + demo message  
view_db.py - prints raw DB contents for screenshots  
conftest.py - lets pytest import the modules  
requirements.txt  
README.md  
USER_GUIDE.md  
REPORT_DRAFT.md  
screenshots_needed.md  
.gitignore  
tests/  
test_auth.py  
test_crypto.py  
test_integrity_attack.py  
screenshots/
```

Source

code

repository:

<https://github.com/ashok-cse/Secure-Messaging-System-Using-Encryption-Techniques>

5.2 User Authentication

The authentication module allows new users to register and existing users to log in. Plaintext passwords are never stored. For every user, a random salt is generated and the password is hashed using PBKDF2-HMAC-SHA256. During login, the same process is repeated and the generated hash is compared with the stored hash.

password + random salt -> PBKDF2-HMAC-SHA256 -> stored password hash

Each user has a unique salt.

Only the salt and password hash are saved in the database.

Failed and successful login attempts are recorded in the security logs.

The active user is stored in the CLI session until logout.

5.3 Symmetric Encryption

The messaging module encrypts every plaintext message before storing it. The implementation uses AES-GCM from the Python cryptography library. AES-GCM provides confidentiality and authenticated encryption. A new random nonce is generated for every message to prevent repeated encryption patterns.

plaintext message -> AES-GCM encryption -> ciphertext + nonce

The AES key is not hardcoded.

The sender and receiver enter a shared chat secret.

PBKDF2-HMAC-SHA256 derives the encryption key from the shared secret and random salt.

The shared secret is never stored in the database.

Only encrypted ciphertext, nonce, salt, and HMAC are stored.

5.4 Message Integrity Using HMAC-SHA256

To detect unauthorized modification, the system calculates an HMAC-SHA256 value over the encrypted message and important metadata. Before decryption, the HMAC is recalculated and compared. If the values do not match, the application stops decryption and reports a possible tampering attack.

HMAC data = sender + receiver + timestamp + nonce + salt + ciphertext

HMAC verification is performed before decryption.

Modified ciphertext is detected immediately.

Failed verification is logged as a security event.

The receiver is warned that the message may have been changed.

5.5 Database Design

Table	Main Fields	Description
users	id, username, password_hash, salt, created_at	Stores registered users and secure password data.
messages	id, sender, receiver, timestamp, nonce, salt, ciphertext, hmac, tampered	Stores encrypted messages only (no plaintext). The 'tampered' flag is set when the attack simulation modifies a message.
logs	id, timestamp, event_type, username, details	Stores security and system events.

6. ATTACK SIMULATION

6.1 Man-in-the-Middle Message Tampering

The MITM simulation demonstrates what happens when an attacker changes an encrypted message after it has been created. The attack module modifies one byte of the stored ciphertext or HMAC. When the receiver tries to open the message, the HMAC verification fails.

Step	Action	Expected Result
1	Sender sends encrypted message.	Message is saved as ciphertext.
2	Attack simulation modifies the ciphertext/HMAC.	Encrypted data is no longer authentic.
3	Receiver tries to decrypt the message.	HMAC verification fails.
4	Application blocks decryption.	Tampering warning is shown and logged.

6.2 Brute-Force Login Attempt

The brute-force simulation repeatedly attempts to log in using incorrect passwords. The system rejects the attempts and records each failure in the security logs. This demonstrates monitoring of suspicious authentication activity.

Incorrect passwords are not accepted.

Repeated login failures are logged.

The log file helps identify brute-force behavior.

The system can be extended to temporarily lock accounts after a defined number of failed attempts.

7. LOGGING SYSTEM

The logging module records important events in the SQLite logs table. These logs help the user or administrator review application activity and detect security problems.

User registration.

Successful login.

Failed login.

Message encryption.

Successful message decryption.

Failed HMAC verification.

Failed decryption attempt.

MITM/tampering attack simulation.

Brute-force login simulation.

8. RESULTS

The implemented system successfully demonstrates the main security requirements of the homework.

Users can register and log in securely.

Passwords are protected using salted PBKDF2-HMAC-SHA256 hashing.

Messages are encrypted using AES-GCM before storage.

Plaintext messages are not stored in the database.

Message modification is detected using HMAC-SHA256.

Decryption is blocked when integrity verification fails.

MITM/tampering attack simulation works as expected.

Brute-force login attempts are rejected and logged.

Security logs are available from the CLI menu.

8.1 Automated Testing (Pytest)

A suite of automated tests was written using pytest to verify the security functions. The tests cover password hashing and verification (test_auth.py), AES-GCM encryption/decryption including failure on a wrong shared secret (test_crypto.py), and HMAC integrity with tampering detection (test_integrity_attack.py). All 19 tests pass, confirming that encryption, decryption, and tamper detection behave correctly.

```

===== test session starts =====
platform darwin -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/ashokcse/Secure_Messaging_System_Using_Encryption_Techniques/secure_messaging_system/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/ashokcse/Secure_Messaging_System_Using_Encryption_Techniques/secure_messaging_system
collecting ... collected 19 items

tests/test_auth.py::test_hash_password_is_deterministic_with_same_salt PASSED [ 5%]
tests/test_auth.py::test_hash_password_differs_with_different_salt PASSED [ 10%]
tests/test_auth.py::test_verify_password_correct_and_wrong PASSED [ 15%]
tests/test_auth.py::test_register_and_login_success PASSED [ 21%]
tests/test_auth.py::test_register_duplicate_username_fails PASSED [ 26%]
tests/test_auth.py::test_login_wrong_password_fails PASSED [ 31%]
tests/test_auth.py::test_login_unknown_user_fails PASSED [ 36%]
tests/test_auth.py::test_plaintext_password_is_never_stored PASSED [ 42%]
tests/test_crypto.py::test_encrypt_then_decrypt_roundtrip PASSED [ 47%]
tests/test_crypto.py::test_ciphertext_is_not_plaintext PASSED [ 52%]
tests/test_crypto.py::test_each_message_uses_fresh_nonce_and_salt PASSED [ 57%]
tests/test_crypto.py::test_wrong_secret_fails_to_decrypt PASSED [ 63%]
tests/test_crypto.py::test_wrong_aad_fails_to_decrypt PASSED [ 68%]
tests/test_crypto.py::test_key_derivation_splits_into_two_distinct_keys PASSED [ 73%]
tests/test_integrity_attack.py::test_hmac_verifies_for_untampered_message PASSED [ 78%]
tests/test_integrity_attack.py::test_tampering_ciphertext_is_detected_by_hmac PASSED [ 84%]
tests/test_integrity_attack.py::test_tampering_hmac_is_detected PASSED [ 89%]
tests/test_integrity_attack.py::test_tampered_ciphertext_fails_to_decrypt PASSED [ 94%]
tests/test_integrity_attack.py::test_attack_on_empty_inbox_reports_nothing_to_attack PASSED [100%]

===== 19 passed in 1.70s =====

```

9. ADVANTAGES

Simple and easy-to-understand CLI implementation.

Strong symmetric encryption using AES-GCM.

Secure password storage with salted hashing.

HMAC-based message integrity verification.

Tampering detection before decryption.

No hardcoded encryption keys.

Modular code structure.

Useful security event logging.

10. LIMITATIONS

The application runs locally and is not deployed as a network server.

The shared chat secret must be exchanged outside the application.

No public-key cryptography or certificate-based key exchange is implemented.

No multi-factor authentication is included.

The CLI interface is simple and not as user-friendly as a full GUI.

Screenshots must be captured manually after running the project.

11. FUTURE ENHANCEMENTS

Add a graphical or web-based interface.

Add RSA or Diffie-Hellman key exchange for safer key sharing.

Add multi-factor authentication.

Add account lockout after repeated failed login attempts.

Add real-time client-server messaging.

Add digital signatures for stronger non-repudiation.

Add encrypted backup and export features.

Deploy the project as a secure web service.

12. SCREENSHOTS

The screenshots below were captured from the actual implemented CLI application (rendered from real program output). They demonstrate each core feature of the Secure Messaging System in action.

1. User Registration in CLI

```
Select an option [1-8]: 1

--- Register ---
Choose a username: charlie
Choose a password: *****
Confirm password: *****
Registration successful.
```

2. Successful Login

```
Select an option [1-8]: 2

--- Login ---
Username: charlie
Password: *****
Login successful.
Welcome, charlie!
```

3. Sending an Encrypted Message

```
Select an option [1-8]: 3

--- Send Encrypted Message ---
Receiver username: bob
Shared chat secret (agreed with receiver): *****
Message: Meet me at 6pm by the library.

Message encrypted and sent to 'bob'. (id=2)
Stored ciphertext (base64): Ug9gh3eQswYAqv/F9XCvHb4ij0mYbEy2qxFcIBFhcX9i5YKo13el4mdd5Uirfw==
```

4. Encrypted Message Stored in Database


```
$ python view_db.py

===== USERS (no plain passwords) =====
alice      salt=863d5a0c2090... hash=3bc6b7812de58daa...
bob        salt=56185e5eaae9... hash=28de1a0cdd469d0a...

===== MESSAGES (only ciphertext stored) =====
id=1 alice->bob
ciphertext=Xdowjgqq4EtVc0iixgUtayE5zB/UkqahwvB00RKRvAUz94JdAEs4++aBKt87ZQpXrUG8r5MU7+oT3Z6EmIwgp2ytYH4=
hmac       =U/7nzjvVNwBAX1Dg1+jebNn0Nhh4uyN7WLUMg7Vit5s=
```

5. Receiver Inbox and Successful Decryption

```
Select an option [1-8]: 4

--- Inbox ---
You have 1 message(s).
Shared chat secret to decrypt your messages: *****

-----
Message id=1 from 'alice' at 2026-06-21 16:59:01
>> Decrypted message: Hello Bob, this is a secret demo message from Alice!
```

6. Man-in-the-Middle Tampering Simulation

```
Select an option [1-8]: 5

--- Attack Simulation (MITM / Tampering) ---
This will modify the latest message in YOUR inbox to simulate
an attacker changing the data in transit.
  1. Tamper with the ciphertext
  2. Tamper with the HMAC tag
Choose target [1/2]: 1
Attack complete. The ciphertext of message id=1 (from 'alice') was modified.
Ask 'bob' to open their inbox to see the integrity check fail.

Now choose option 4 (View inbox) to see the detection.
```

7. Integrity Check Failure

```
Select an option [1-8]: 4
```

```
--- Inbox ---
```

```
You have 1 message(s).
```

```
Shared chat secret to decrypt your messages: *****
```

```
-----  
Message id=1 from 'alice' at 2026-06-21 16:59:01 (TAMPERED in DB)  
>> Integrity check failed. Message may have been modified.
```

8. Brute-Force Login Detection

Select an option [1-8]: 2

--- Login ---

Username: bob

Password: *****

Invalid username or password.

Select an option [1-8]: 2

--- Login ---

Username: bob

Password: *****

Invalid username or password.

Select an option [1-8]: 2

--- Login ---

Username: bob

Password: *****

Invalid username or password.

Select an option [1-8]: 6

--- Security Logs ---

===== SECURITY LOGS (newest first) =====

TIMESTAMP	EVENT	USER	DETAILS
2026-06-21 17:10:13	LOGIN_FAILURE	bob	Login failed: wrong password
2026-06-21 17:10:13	LOGIN_FAILURE	bob	Login failed: wrong password
2026-06-21 17:10:13	LOGIN_FAILURE	bob	Login failed: wrong password
2026-06-21 17:10:13	REGISTER	bob	New user registered
2026-06-21 17:10:13	REGISTER	alice	New user registered

9. Security Logs Display

Select an option [1-8]: 6

--- Security Logs ---

===== SECURITY LOGS (newest first) =====

TIMESTAMP	EVENT	USER	DETAILS
2026-06-21 17:03:52	INTEGRITY_FAILURE	bob	HMAC verification failed for message id=1
2026-06-21 17:03:52	ATTACK_SIMULATION	bob	MITM/tampering attack: modified ciphertext of message id=1 from 'alice' to 'bob'
2026-06-21 17:03:52	DECRYPT_SUCCESS	bob	Decrypted message id=1
2026-06-21 17:03:51	LOGIN_SUCCESS	bob	Login successful
2026-06-21 17:03:51	REGISTER	bob	New user registered
2026-06-21 17:03:51	REGISTER	alice	New user registered

13. USER GUIDE

13.1 Installation

```
pip install -r requirements.txt
```

13.2 Run Demo Data

```
python demo_data.py
```

13.3 Start Application

```
python app.py
```

13.4 Basic Usage Steps

Start the application from the terminal.

Register two users, for example alice and bob.

Log in as alice.

Send an encrypted message to bob using a shared chat secret.

Log out and log in as bob.

Open the inbox and enter the same shared chat secret to decrypt the message.

Run the attack simulation option to modify the encrypted message.

Try to decrypt the tampered message and observe the integrity failure warning.

Open security logs to verify recorded events.

14. CONCLUSION

The Secure Messaging System successfully implements the required software security concepts using Python. The system protects user passwords using secure hashing, encrypts messages with AES-GCM, verifies message integrity using HMAC-SHA256, and detects tampering attempts before decryption. The attack simulations demonstrate that modified encrypted data cannot be accepted as valid. The logging module further supports monitoring of login attempts, encryption events, failed verification, and attack detection. Overall, the project satisfies the core goals of confidentiality, integrity, authentication, and security monitoring in a clear and practical implementation.